

[L0-L1 Core Essence & Design Logic]

**L0 Essence:** A monolithic macrokernel initializing from real-mode assembly bootloaders, transitioning to 64-bit long-mode page tables, managing memory frames through a 2-power Buddy allocator and slab object caches, scheduling process execution via virtual runtime red-black trees, and abstracting I/O devices via VFS page caches.

L1 Logic:

- Bootstrapping & Initialization** (M1) shifts from assembly-level `head_64.S` initial paging to C-language `start_kernel()` subsystem initializations and architecture setup.
- IDT Traps & CFS Scheduler** (M2) handles interrupts through an IDT table mapping assembly contexts, scheduling processes fairly via rb-tree virtual runtimes.
- Buddy System Memory Allocator** (M3) divides physical RAM into numa nodes and zones (DMA, Normal), allocating pages in power-of-two blocks to prevent fragmentation.
- Slab & Page Cache I/O** (M4-M5) aggregates small objects on top of buddy pages in lock-free cpu caches, caching block writes via dirty page caches.

[L2 Linux Kernel Data Flow Topology]

• [Assembly Boot head\_64.S] → [C Entry start\_kernel()] → [E820 RAM Table Query] → [Buddy Free List Init] → [SLUB Object Cache Setup] → [CFS Scheduler Boot] → [VFS Mount] → [Page Cache Read] → [BIO Request Disk Flush]

[M1.1] x86 Boot Sector Load and Transition to 64-bit Long Mode (Boot to Long Mode)

- **Bootloader Load:** The BIOS/UEFI reads the Master Boot Record, executes the GRUB bootloader, copies the kernel bzimage to RAM, and jumps to the entry point in `boot/header_5`.
- **Long Mode Transition:** After checking CPU registers in 16-bit real mode, the kernel sets the PE bit in CR0 to transition to 32-bit protected mode. It then configures initial page tables, sets the PAE bit in CR4, and enables long mode in the EFER register.

[M1.2] head\_64.S Assembly Entry and C start\_kernel (head\_64.S to start\_kernel)

- **Assembly Initialization:** The CPU jumps to `arch/x86/kernel/head_64.S` in 64-bit mode. The assembly code clears the BSS segment, configures CPU segment registers, allocates a temporary boot stack, and loads global descriptor table (GDT) pointers.
- **C Entry Jump:** After completing assembly setup, the kernel executes a jump instruction to enter the C entry point: `start_kernel()`.

[M1.3] setup\_arch and E820 Physical RAM Table Mapping (setup\_arch & E820)

- **E820 Address Map:** The function `setup_arch()` parses `boot_params` from the bootloader, scanning the E820 map to identify usable and reserved physical memory regions.
- **Memblock Allocator:** The kernel uses a temporary allocator (`memblock`) to manage physical memory allocations before the primary page allocator initialization completes.

[M1.4] init\_IRQ and Formal Page Table Handover (init\_IRQ & Kernel Page Tables)

- **IDT Initialization:** The function `init_IRQ()` initializes the Interrupt Descriptor Table (IDT) with system call handlers and hardware interrupt gates.
- **Page Table Handover:** The kernel swaps out temporary boot page tables for the formal `swapper_pg_dir` page table, mapping virtual memory addresses for subsequent allocations.

[M2.1] IDT Entry Descriptors and Interrupt Frame Push (IDT & Interrupt Stack)

- **IDT Gate Layout:** The IDT contains 256 gate descriptors mapping interrupt vectors to their corresponding Interrupt Service Routine (ISR) addresses.
- **Hardware Stack Push:** When an interrupt occurs, the CPU pushes current SS, RSP, RFLAGS, CS, and RIP registers onto the kernel stack and redirects execution to the entry gate in `common_interrupt`.

[M2.2] syscall Instruction Vector and sys\_call\_table Dispatch (syscall & sys\_call\_table)

- **syscall Instruction:** 64-bit system calls use the `syscall` instruction instead of software interrupts. The CPU reads the `IA32_LSTAR` MSR register to redirect execution to `entry_SYSCALL_64`.
- **Call Dispatch:** The entry routine pushes registers onto the stack and uses the call ID in RAX to look up and invoke the system call function in `sys_call_table`.

[M2.3] Softirqs and Workqueues Bottom Half Processing (Bottom Halves)

- **Bottom Half Processing:** To prevent hard interrupts from stalling the CPU, interrupt tasks are split. The top half executes minimal register saves and registers a softirq.
- **Asynchronous Engines:**
  - 1. softirq:** Runs in interrupt context and cannot sleep; ideal for fast network interface packet processing.
  - 2. workqueue:** Executes in process context using background threads; can perform I/O operations and sleep.

[M2.4] CFS Virtual Runtime Calculations and RB-Tree Sorting (CFS vruntime Update)

- **Virtual Time Progress:** The CFS scheduler tracks CPU allocation using a `vruntime` metric. When a process runs, its virtual runtime increases:  $\Delta vruntime = \Delta physical\_time \times \frac{NICE\_0\_LOAD}{weight}$ .
- **RB-Tree Ordering:** The scheduler organizes runnable processes in a red-black tree sorted by `vruntime`. It selects the leftmost node (`rb_leftmost`) for execution in  $O(1)$  time.

[M2.5] switch\_to Context Switch Stack Pointer Redirect (switch\_to Context Switch)

- **Context Switch:** When the scheduler switches tasks, it invokes `switch_to()`.
- **Stack Redirection:** The routine updates the RSP register to point to the target thread's stack (`thread.sp`), restoring register states from the target stack to complete the context switch.

[M3.1] NUMA Nodes and Memory Zone Allocation Boundaries (NUMA Zones)

- **NUMA Nodes:** On multi-socket systems, physical memory is partitioned into NUMA nodes (represented by `pg_data_t`) close to their respective CPU sockets to minimize memory access latencies.
- **Memory Zones:** Nodes divide memory into zones:
  - 1. ZONE\_DMA:** The first 16MB of physical RAM, allocated for legacy DMA devices.
  - 2. ZONE\_NORMAL:** Memory from 16MB to 4GB, directly mapped by the kernel.
  - 3. ZONE\_HIGHMEM:** Memory above 4GB (used on 32-bit systems; obsolete on 64-bit platforms).

[M3.2] struct page Descriptor Meta Allocation (struct page Layout)

- **Page Frame Mapping:** The kernel allocates a `struct page` descriptor in the global `mem_map` array for every 4KB physical page frame.
- **Reference Tracking:** Descriptors track page state using `_refcount` (kernel references) and `_mapcount` (user 页表 mappings). Pages return to the free list when `_refcount` reaches 0.

[M3.3] Buddy System Power-of-Two Allocator and Merging (Buddy Allocator)

- **Free Area Arrays:** The Buddy system manages free memory blocks in a `free_area` array indexed by orders (from 0 to 10).
- **Splitting and Merging:**
  - **Allocation:** Requests search for available blocks at the target order. If empty, the allocator splits a higher-order block.
  - **Release:** Releasing blocks calculates the buddy address:  $Buddy = Block \oplus (Order \ll PAGE\_SHIFT)$ . If the buddy is free, it merges them into a higher-order block.

[M3.4] Memory Compaction and Page Migration (Memory Compaction)

- **Fragmentation:** Over time, memory allocation patterns scatter page fragments across the address space, preventing the allocation of large contiguous blocks (e.g., 2MB pages).
- **Compaction Scans:** The compaction thread (`kcompactd`) scans the zone from both ends: migrating active pages from one end to free blocks at the other, creating large contiguous free regions.

[壹] Linux Kernel Core Trade-offs & Fallacies Matrix

[M4.1] Slab Caches and Kernel Object Recycling (Slab Allocator Concept)

- **Fine-Grained Allocations:** The Buddy allocator works with 4KB pages. Allocating small kernel objects (e.g., file descriptors) directly from pages causes memory fragmentation.
- **Object Recycling:** Slab caches pool pre-allocated, fixed-size objects in memory. Released objects are marked "free" but remain initialized, reducing CPU overhead during subsequent allocations.

[M4.2] kmem\_cache CPU and Node Layering (kmem\_cache Hierarchies)

- **Per-CPU Fast Path:** `kmem_cache` allocates objects using a fast-path cache (`kmem_cache_cpu`) to bypass locking overhead on a per-CPU basis.
- **Node Shared Path:** If the local CPU cache is depleted, a slow-path routine locks and fetches objects from the NUMA node's shared cache (`kmem_cache_node`).

[M4.3] SLUB Allocator Inline freelist Offset (SLUB freelist Offset)

- **Zero-Metadata Allocation:** The SLUB allocator simplifies memory management by removing metadata structures.
- **Inline Pointers:** It stores the next free object pointer (`freelist`) directly inside the unused object space, eliminating metadata overhead.

[M4.4] kmalloc Size-Based Route Allocation (kmalloc Implementation)

- **Pre-Allocated Sizes:** The function `kmalloc()` manages general-purpose dynamic memory allocations.
- **Slab Routing:** The kernel pre-allocates slab caches for common sizes (e.g., `kmalloc-32`, `kmalloc-128`, `kmalloc-2048`). Calls to `kmalloc(100)` route requests to the nearest size (e.g., `kmalloc-128`).

[M5.1] Page Cache Layout and Radix Tree/XArray Indexing (Page Cache Layout)

- **Memory Mirrored Files:** The page cache caches file data in physical memory page frames.
- **Tree Indexing:** Each inode's `address_space` indexes its cached pages using a Radix Tree (or an XArray in modern kernels), enabling  $O(1)$  page lookups by file offset.

[M5.2] Page Writeback Threads and Dirty Page Limits (Page Writeback Control)

- **Dirty Page Buffering:** Modifying page cache data marks pages as dirty, delaying disk writes.
- **Writeback Thresholds:** Background threads (e.g., `wb_workfn`) trigger writebacks. If dirty pages exceed `vm.dirty_background_ratio`, they run asynchronously; if they exceed `vm.dirty_ratio`, writes block until pages flush.

[M5.3] Page Frame Reclamation Algorithm (PFRA) and LRU Lists (Page Reclaim PFRA)

- **Low Watermark Reclaim:** When free memory falls below the zone's low watermark, `kswpd` triggers page reclamation.
- **LRU List Migrations:** Zones partition pages into Active and Inactive LRU lists. Unused pages migrate to the Inactive list for reclamation.

[M5.4] LRU Replacement Scanning and Swappiness Controls (LRU & Swappiness)

- **Two-Pass Scan:** Page reclamation clears the Referenced bit of inactive pages. If the page is accessed before the next scan, it returns to the Active list; otherwise, it is reclaimed.
- **Swappiness Weight:** The `swappiness` value adjusts the ratio of anonymous page swaps to file cache page purges.

[M5.5] OOM-Killer badness() Scoring Algorithm (OOM Killer Scoring)

- **Out-of-Memory Recovery:** If the kernel runs out of memory and cannot reclaim pages, it triggers the OOM killer.
- **Score Calculations:** The function `badness()` scores active processes:  $Points = \frac{RSS\ Memory}{Total\ System\ Memory} \times 1000$ . The OOM killer terminates the process with the highest score (modified by `oom_score_adj`) to free memory.

[M5.6] Swap Space Management and Page Faults (Swap Page Management)

- **Anonymous Page Swap:** The kernel swaps anonymous memory pages to dedicated disk swap partitions.
- **Swap Map Tracking:** The kernel tracks swap slot allocations using `swap_map`. Accessing a swapped page raises a page fault, triggering the kernel to read the page back to RAM.

[M6.1] VFS Core Metadata Objects Topology (VFS Core Objects)

- **VFS Components:**
  1. **super\_block:** Tracks file system metadata (e.g., block sizes, mount points).
  2. **inode:** Represents a physical file on disk and contains data block pointers.
  3. **dentry:** Maps filenames to inode numbers to accelerate path resolutions.
  4. **file:** Tracks open file descriptors and current seek offsets.

[M6.2] file\_operations VFS Driver Redirections (file\_operations VFS)

- **Virtual Operations:** The `struct file` includes a pointer to a `file_operations` structure containing function vectors.
- **Driver Mounting:** Calls to `read()` resolve to `file->f_op->read()`, routing operations to file-system-specific code (e.g., `ext4_file_read` or a character device driver).

[M6.3] BIO Request Packets and bio\_vec Mappings (struct bio Layout)

- **I/O Transfer Units:** The VFS translates file page operations into `struct bio` packets for block devices.
- **Scatter-Gather DMA:** A BIO contains an array of `bio_vec` segments mapping physical pages, offsets, and lengths, allowing DMA controllers to transfer non-contiguous blocks.

[M6.4] I/O Schedulers and Request Queue Merging (I/O Scheduler queue)

- **Request Merging:** BIOs enter a `request_queue`. Schedulers merge contiguous sector requests (Elevator Merge) into single requests to reduce disk head movements.
- **Multi-Queue Scheduling:** The `blk-mq` framework allocates queues to CPU cores, using schedulers like BFQ or Kyber to bypass global lock contention on fast devices.

[M6.5] mmap Address Mapping and Page Fault Read (mmap & Page Fault)

- **Virtual Mappings:** The call `mmap()` creates a `vm_area_struct` mapping virtual addresses to file indexes without allocating physical pages.
- **Zero-Copy Loading:** Accessing mapped memory triggers a page fault. The kernel reads the file segment into the page cache and maps its physical address to the process's page table, avoiding copy overhead.

🔧 Zone T: Linux Kernel Diagnostics & Tuning Reference Laboratory

T1 Linux Kernel Tuning Parameters

`\texttt{--- (---) \texttt{kernel.sched_latency_ns (12000000 - 24000000) \texttt{CFS scheduling target latency. Ensures all runnable processes run at least once within this period. \texttt{kernel.sched_wakeup_granularity_ns (2000000 - 4000000) \texttt{The vruntime difference required for a newly awakened process to preempt the currently running process. \texttt{vm.dirty_background_ratio (5 - 10) \texttt{The percentage of system memory that can contain dirty pages before background flush threads start flushing them to disk. \texttt{vm.dirty_ratio (20 - 30) \texttt{The max percentage of memory containing dirty pages. Reaching this limit forces user writes to block until pages are written to disk.`

T2 Linux Kernel Tracing Commands

`\textbf{1. Monitor Slab Cache Object Allocations} \texttt{\# Print real-time statistics for active slab caches (e.g., dentries, inodes, kmalloc pools) to trace memory leaks \texttt{slabtop -o -s a} \textbf{2. Trace Kernel Functions Using Ftrace Graph} \texttt{\# Enable the ftrace function graph tracer on the page allocator entry point \texttt{\_alloc_pages} \texttt{echo function_graph > /sys/kernel/debug/tracing/current_tracer} \texttt{echo \_alloc_pages > /sys/kernel/debug/tracing/set_graph_function} \texttt{\# /sys/kernel/debug/tracing/trace | head -n 40} \textbf{3. Monitor Real-Time CPU Kernel Function Hotspots} \texttt{\# Profile CPU usage across kernel functions (at 99Hz) to identify locking bottlenecks and hot code paths} \texttt{perf top -g}`